CONCEPT MAPPING

Map the evolution of language models - for early statistical approaches to modern Large Language models. Connect language modeling, neural networks, training processes, and practical applications to demonstrate why Generative AI can create new content - while discriminative models primarily perform prediction or classification.

Evolution of Language Models

Statistical - Approaches (1990s - early 2000s)
- -> used probability & statistics
- -> example :- N-gram models

Language Modelling (2000s)
- -> predicts the next word in a sequence.
- -> shift from simple n-grams to probabilistic model.

Neural - Network (2000s - 2010s)
- -> Learn word representations (embeddings).
- -> Improve accuracy

Sequence-Modelling (2010s)
- -> Handle sequential data
- -> RNNs and LSTMs capture context

Transformers (2017)
- -> uses self-attention Mechanism
- -> processes all words in parallel

Self Attention (2017)
- -> Focuses on important words in input.
- -> captures long-range context.

Foundation Models (2018 - present)
- -> Large scale pre-trained models on massive data.
- -> General-purpose and adaptable.

Generative AI (2020s - present)
- -> uses foundation models to generate text, images, codes etc...,

Modern Large Language Models (LMMs) (2020s - Present)
- -> Billions of parameters
- -> Advanced reasoning and instruction-following
- -> Ex :- GPT-4, Gemini

Key Transition

Statistics -> Language Modeling -> Neural Networks -> Sequence Modeling -> Transformers -> Self-Attention -> Foundation Models -> Generative AI -> LLMs

CONCEPT MAPPING

Construct a concept map explaining how Transformer based language model processes a text sequence. Connect a tokenization, embeddings, positional information, self attention, neural network, transformations, probability estimation, and next-token generation to illustrate the complete processing pipeline.

Transformer-Based Language Model Processing Pipeline

1. Tokenization

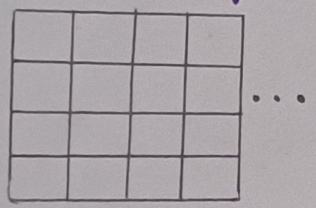
Text :- "I love AI"

↓

[I] [LOVE] [AI]

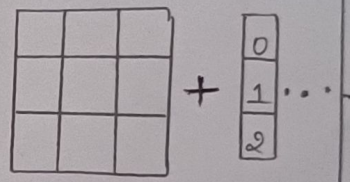
-> Split's text into tokens (words / subwords)

2. Embeddings



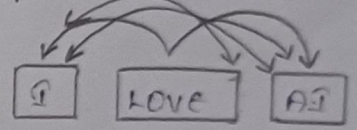
-> Convert's tokens into vector representation (embeddings).

3. Positional Information



-> Add's position values to keep word order information.

4. Self-Attention



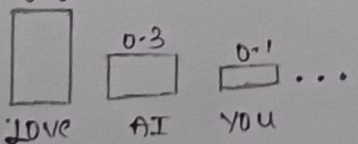
-> Find's important word's in the sequence.

5. Neural-network Transformations

Feed Forward Network

* Processes the attended information through multiple layer's

6. Probability estimation



-> convert's output to probabilities for next token

7. Next-Token Generation

Output: love

-> select the most likely token and generates it.